



WHITE PAPER

The Sovereign Communication & Permissions Architecture

*Zero-identity communication and non-briable data stewardship:
removing the intermediary's access instead of asking for its re-
straint.*

GHOSTBOX PROTOCOL

Version 0.1.0 · Working Draft

May 2026

Cory A. Ottenwess

Companion text:

Notes from an Acceleration Native

Abstract

Modern communication and personal-data platforms are built on one structural fact: the operator sits between people and monetizes the position. The social graph, the message stream, and the behavioral record are the assets, and no privacy policy changes who holds them. This paper specifies an alternative in which the operator is removed from the value path by construction rather than by promise.

The architecture applies a single design decision at two layers. The **GhostBox Protocol** removes the transport operator's ability to learn who is talking to whom. The **Companion Permissions Layer** removes any intermediary's ability to extract value from a personal data vault. In both cases the guarantee is mathematical or economic structure, not declared good behavior.

What follows is the argument and the shape of the system: the problem it answers, the principles it holds to, the three-layer GhostBox stack for identity, transport, and discovery, the companion model that turns a relationship gradient into vault access control, and an honest accounting of what the design does not solve. A companion technical reference specifies the normative requirements and the corrected reference implementation.

Contents

1	The Extraction Architecture	3
2	The Thesis: Remove Access, Not Trust	4
3	Design Principles	5
4	The Spectre Stack	6
5	Identity Without Registration	7
6	Transport That Cannot Link	8
7	Presence as a Gradient	9
8	The Companion as Gatekeeper	10
9	Threat Model and Honest Limitations	11
10	Sustainability Without Surveillance	12
11	Status and Reference Implementation	13

1. The Extraction Architecture

Every dominant communication platform of the last two decades shares a shape. A company stands between people who want to reach each other and charges for the position, not in money taken from the participants but in access taken from them. The product is delivered free because the participants are not the customers. The customers are whoever will pay to reach, profile, or influence them, and the asset being sold is the record of who knows whom, who said what, and what each person is likely to do next.

This is not a failure of the platforms. It is their design working as intended. The intermediary position is valuable precisely because it is comprehensive: the operator sees the full graph, the full stream, and the accumulating behavioral surplus that no single participant can see about themselves. A messaging service learns the social network. A data platform learns the person. The economics follow the visibility, and the visibility is structural.

The standard response to this is the privacy policy, which is a promise about behavior. A platform that holds the graph promises not to misuse it. A service that holds the vault promises to ask before sharing. These promises are worth exactly as much as the incentive to keep them, and the incentive runs the other way. When the revenue model rewards access, restraint is a cost, and costs erode. The Replika enforcement action in Italy in 2025 is one documented instance of the predictable result: a companion product that required user data to live in operator-usable form, and used it accordingly.

The question is not whether a given operator will behave. It is whether the architecture makes misbehavior possible at all.

That reframing is the whole move. As long as the operator can see the graph, the only defense is trust, and trust at scale is unenforceable. The alternative is to build systems where the operator cannot see the graph, cannot read the content, and cannot reach the vault, so that there is nothing to promise about because there is nothing to misuse. That is harder to build and it is the only defense that holds when the incentives turn.

2. The Thesis: Remove Access, Not Trust

The architecture rests on one decision applied in two places: **remove the intermediary's ability to extract value from a user's data by removing the intermediary's access to that data.** Stated that plainly it sounds obvious. The work is in making it true at the level of cryptography and economics rather than at the level of policy.

Applied to message transport, the decision produces the GhostBox Protocol, in which the server that moves messages is structurally unable to learn who is communicating with whom. Applied to personal data, the same decision produces the Companion Permissions Layer, in which the agent that mediates access to a person's vault has no economic relationship with anyone seeking that access and therefore nothing to gain by granting it. Either piece is useful alone. Together they close the loop: private transport for the messages, non-briable stewardship for the data the messages are about.

This is deliberately an architectural argument, not an adversarial one. The aim is not to defeat platforms in a contest but to build infrastructure where the contested resource, comprehensive access, does not accumulate in any one place to begin with. The conflict that looks like people versus technology is better understood as a question of who holds the structure, and structure can be built differently before it hardens.

SCOPE

This paper specifies the boundary, not the contents. The personal data vault's storage format and the companion's reasoning model are separate prerequisites with their own specifications. What is defined here is how the world reaches the vault and how the vault reaches the world.

3. Design Principles

Six principles are normative. Every requirement in the technical reference traces back to one of them, and every design tradeoff is resolved in their favor.

P1 – POSSESSION IS IDENTITY

Identity is derivable by the user from secrets and factors they hold, with no registration step and no server-held account record. There is nothing for a platform to revoke, suspend, or sell, because no platform holds it.

P2 – THE SERVER CANNOT BUILD THE GRAPH

The transport infrastructure is structurally incapable of linking sender to recipient. This is a mathematical property of the design, not a policy promise. The social graph must not exist in the server's data in any recoverable form.

P3 – EXTRACTION INCENTIVE ABSENT BY CONSTRUCTION

No component that mediates access to user data has a financial relationship with parties seeking that data. The gatekeeper cannot be bribed because the revenue model that would reward letting someone in does not exist.

P4 – TRUST IS VERIFIABLE, NOT DECLARED

Every component that handles user data is auditable: open code, public governance, behavior checkable against the specification. Trust is earned through inspection, the way a contract is trustworthy because its terms are written down and deviation is detectable.

P5 – PRESENCE MIRRORS SOCIAL REALITY

The relationship model represents the full gradient of human connection, not a connected-or-not binary. Access is scoped, revocable, and user-defined at every stage.

P6 – THE USER DECIDES; THE COMPANION HOLDS

The companion does not advance trust, widen access, or alter a permission on its own authority. It maintains the shape of what the user has decided, flags when that shape may need revisiting, and waits for affirmation before anything changes.

4. The Spectre Stack

GhostBox is organized as three independent layers. Independence is a hard requirement: compromise of one layer must not compromise another. The layers move from the invisible and private, through the passive and unlinkable, to the voluntary and present.

Corporeal Layer **DISCOVERY**

Voluntary presence. How parties find each other on terms they set. Public Mask, Lobby, and Ghost Channel components; four discovery modes; the five-state relationship model.

Specter Layer **TRANSPORT**

A passive dead-drop. Deposit to a Locator Hash; drain by challenge-response; time-limited storage. The server that cannot link sender to recipient.

Spirit Layer **IDENTITY**

Invisible. Exists only in user memory and transient RAM. A Quad-Key passes through a memory-hard derivation into a public Locator Hash and a private Access Token. No server holds it.

Beneath and beside the stack sit two elements that are not part of GhostBox transport but consume it. The **personal data vault** holds the user's information and is out of scope here. The **Companion Permissions Layer** uses the Corporeal Layer's relationship model as its vocabulary for vault access control and uses the Specter Layer as its transport. The stack provides private plumbing; the companion decides what may flow through it.

5. Identity Without Registration

The Spirit Layer makes identity mathematical rather than administrative. A user composes a **Quad-Key** from four or more high-entropy factors they hold, and a memory-hard derivation function transforms it into two outputs: a public Locator Hash that is the address others deposit to, and a private Access Token that authorizes draining the user's drop-box and decrypting its contents.

Because the identity is derived and never stored, it cannot be seized: there is no physical record to confiscate. It cannot be revoked: there is no administrator to revoke it. And it is device-agnostic: the same Quad-Key re-derives the same identity on any device, with all local state wiped on exit. The factors may be things the user knows, things they are, things they have, or people who confirm them, in any combination, because the derivation cares only about total entropy, not factor type.

WHY THE COMPOSITION IS ITSELF SECRET

An implementation never discloses which factor classes a given identity used. An attacker who does not know whether an identity is two passphrases or a passphrase plus a hardware token plus a social-recovery contact cannot even define the attack surface. The unpredictability of the composition is a security property in its own right.

This same property turns account recovery from a structural inevitability into a user-owned choice. A passphrase-only key offers no recovery: forget the words and the identity is gone. Adding a storable hardware token, a persistent biometric, or a re-attestable credential adds recovery paths. The protocol enforces an entropy floor but never imposes the recovery tradeoff in either direction. The person who holds the key decides how recoverable it is.

6. Transport That Cannot Link

The Specter Layer is a dead-drop. The server is a passive storage facility, not a post office. It does not route, and it does not know who left a blob or who will collect it. A sender encrypts a message to the recipient's public key and deposits it at the recipient's Locator Hash. The server stores opaque bytes and starts a retention clock. Later, the recipient proves authorization to drain that address through a challenge-response exchange, and the server releases the blob.

The unlinkability is the point, and it is enforced by what the server is prevented from recording. It sees a deposit. Separately, it sees a challenge-response drain. It is structurally unable to record that the depositor and the drainer are in a relationship, because deposit and drain are independently unlinkable in its records. No sender authentication is required or collected, so there is no sender identity to associate with anything. This is principle P2 realized as plumbing: the graph is not protected by a rule against logging it, it is never constructed.

SPAM WITHOUT SENDER IDENTITY

If anyone may deposit to any address, what stops flooding? The answer lives one layer up. In-bound connection requests land in a disposable, rotatable filtering address called the Lobby. Flooding it reaches nothing past the filter, though draining the junk still costs the recipient something; a planned anonymous rate-limiting token will make flooding asymmetrically expensive without reintroducing sender identity. Sender-identification, the usual anti-spam lever and the usual privacy leak, is replaced by a filter the user controls.

The dead-drop trades real-time delivery for metadata privacy. Retrieval is by polling rather than push, which is less efficient and, for most uses, an acceptable price for an operator that cannot build the graph. The same transport carries more than messages: asset transfers, the encrypted channel-address handoff during a handshake, and zero-knowledge read receipts all ride the same unlinkable path. Receipts are deliberately non-automatic and jittered, so a sender cannot turn them into a covert tracking signal.

7. Presence as a Gradient

If identity is private and device-agnostic, how do two people find each other? The Corporeal Layer answers this without reintroducing a permanent, harvestable identifier. A user has presence only where and when they choose, through four discovery modes, all governed by a five-state model of relationship.

The four modes span the range of how people actually meet. A **public alias** lets anyone who knows it request a connection through the Lobby, suitable for a professional or public surface. A **proximity handshake** lets two users in the same physical place connect with no alias at all, deriving an ephemeral key from shared context and destroying it immediately after. A **mediated introduction** encodes the mutual-friend case, where a third party vouches for both sides without disclosing either identity, and steps out once a direct channel forms. A **relayed social connection** carries an introduction along a chain of existing trust and can grant a temporary, scoped window: a ride from the airport, a week of limited contact, after which the contact returns to the periphery.

The proximity handshake is the strongest privacy posture in the system. A user with no alias, no Lobby, and no listed presence is completely invisible except in the instant they choose to be available. The moment passes and they are invisible again.

7.1 The five-state relationship model

Most platforms collapse human connection into a binary: connected or not, friend or stranger. The five-state model restores the gradient that social reality actually has, and makes every transition the user's to initiate.

State	Channel	Access	Character
Stranger	none	none, no visibility	The default for everyone not admitted through a discovery mode.
Acquaintance	exists, bounded	scoped by time, purpose, or interaction	Holdable indefinitely with no obligation to advance.
Preliminary	full channel, provisional	open but flagged	Either party may step back cleanly, with no demotion notice to the other.
Established	persistent	mutual, at agreed scope	The single state most platforms collapse everything into.
Revoked	destroyed	none, no trace	Not "blocked," which notifies. Revoked is gone: messages simply stop arriving and the other party is not told.

Two design choices in this table carry weight. Demotion never notifies the affected party, because, as the specification puts it, the system does not perform social awkwardness on the user's behalf. And each relationship is independently keyed through its own channel, so revoking one leaks nothing about the others. Connection becomes something a person shapes deliberately rather than a switch the platform flips and broadcasts.

8. The Companion as Gatekeeper

The vault behind the door is the most complete portrait of a person ever assembled in one place. Whatever guards it must be trustworthy in a way that cannot be undermined by incentive. The Companion Permissions Layer specifies a gatekeeper whose job, facing the world, is to refuse, and whose trustworthiness is a matter of structure rather than character.

The **non-briable** property is the load-bearing one. The companion has no financial relationship with any party seeking access to the user's data, and there is no revenue model that rewards granting access. Its only economic relationship is with the user. This is a structural requirement, not a behavioral aspiration: a deployment whose operator can monetize access fails the property regardless of stated policy. Alongside it, the **verifiable** property requires that the companion's code be auditable and its governance public, so that the claim of non-briability can be checked rather than taken on faith.

Access is granted at fine grain. Every grant is specific, tied to a stated purpose, and revocable at any moment. Authorizing one use never implies another, the default scope is the narrowest that satisfies the purpose, and grants carry expiry by default. The companion maintains an access matrix that maps each external party to a relationship state and a permission scope, using the same five-state vocabulary that governs discovery.

The only question the companion brings back, when it brings one at all, is: is this still right?

That restraint is principle P6 made concrete. The companion does not decide how close two people should be, or whether a grant made in an energetic moment still holds once the user is home and thinking clearly. It holds the shape of permissions already given, flags when observation suggests that shape may need revisiting, and waits. Crucially, advancement through the trust phases of the relationship, from training wheels to hands-off to open road, is triggered by the user and never by the platform. A user-initiated audit surface lets the person inspect, correct, prune, and delete the model the companion holds, because agreeing to be modeled is the start of a relationship, not a one-time setup event.

Finally, the companion's reasoning may run on shared compute, but the data it reasons from must never leave the vault in a form the operator can retain or sell. Federated learning, homomorphic encryption, and zero-knowledge architectures are the conformant techniques. A companion that requires user data to sit on its operator's servers in usable form is not a sovereign companion. It is the extraction architecture with a friendlier name.

9. Threat Model and Honest Limitations

The design defends against a specific and stated set of adversaries. A curious or compromised server operator cannot reconstruct the social graph, because deposit and drain are unlinkable by construction. The same operator cannot read content, because payloads are end-to-end encrypted and stored as opaque bytes. An identity thief faces a 128-bit entropy floor, memory-hard derivation, and an attack surface randomized by an unknown factor composition. Coercion or device seizure is met with an optional duress identity and the absence of any local persistence. Future key compromise does not decrypt history, because message encryption uses the forward-secret Double Ratchet construction.

Equally important is what the design does not claim. These limitations are properties of this protocol, named plainly rather than hidden.

- **Polling, not push.** Metadata privacy is bought with delivery latency. For time-critical messaging this is a real constraint.
- **Federation distributes reliability, not just trust.** A node cannot read content or build the graph, but it can go offline or throttle. Federation is a security property, not a reliability guarantee.
- **Biometric factors cannot be rotated.** A leaked biometric hash is compromised forever, which is why a biometric is always one factor among several and never the sole one.
- **Lobby flooding costs the recipient, not the sender.** Unmetered deposits mean a junk flood must be drained and decrypted before the filter discards it; the planned rate-limiting token addresses this without adding a sender identity.
- **Network-layer correlation is out of scope.** IP and timing correlation sit outside the cryptographic guarantees; senders should use an anonymizing network at the transport level. The protocol does not provide this itself.
- **Endpoint compromise defeats everything.** A compromised device is outside what any of this can protect. Client integrity is the deployment's responsibility.

The willingness to state these is itself part of principle P4. A system that asks to be trusted by inspection has to make inspection possible, including inspection of its own edges.

10. Sustainability Without Surveillance

GhostBox collects no data, which means it cannot be funded by advertising or behavioral monetization. That is the point, and it leaves three viable models. **Community hosting:** the server software is open source and anyone can run a federated node, with no central point of control or failure. **Freemium:** a free tier with short retention, with paid tiers offering extended retention, larger transfers, and priority bandwidth, settled through privacy-preserving payment. **Enterprise licensing:** private deployments for organizations where the confidentiality of the communication pattern matters as much as the content, such as legal teams, investigative journalists, civil-society organizations, and medical practices.

Node operators carry obligations that keep the guarantees intact: a node must not log deposit-to-drain linkage or sender addresses, must publish and honor its retention policy, and should publish a reproducible build hash so clients can verify that the code it runs matches the audited source. The funding model and the privacy model are aligned rather than opposed, which is what makes the design durable instead of merely well-intentioned.

11. Status and Reference Implementation

This is a working draft, version 0.1.0, open for implementation and review. A reference implementation accompanies the specification: a canonical TypeScript implementation of the Spirit Layer's identity derivation and a Python reference, validated against each other by frozen cross-language test vectors. The reference code is a readable demonstration of the correct construction, not a hardened library, and it has not yet been independently audited. That fact is stated rather than glossed.

The reference code diverges from the illustrative snippets in the book *Notes from an Acceleration Native* in two deliberate ways: it binds the derivation salt to the Quad-Key composite rather than using a static salt, and it defines a versioned Argon2id parameter ladder rather than a single fixed parameter set. Where the repository and the book disagree, the repository governs for implementers, and the book remains accurate as illustration. These divergences are documented so the difference reads as intent, not contradiction.

Two conformance levels are defined. **Level 1, GhostBox Transport**, satisfies the identity, transport, discovery, and cryptographic requirements. **Level 2, Sovereign Companion Deployment**, satisfies Level 1 and the full Companion Permissions Layer. The accompanying technical reference specifies every normative requirement, the corrected derivation pipeline, the wire protocols, and the implementation API.

We know what it would look like if we built it differently. Naming the architecture precisely, before it hardens, is where every different outcome has had to start.

• • •